Thrive With AI

Problem Setting

This project has three entirely different AI algorithms that take control of life as it evolves from a single organelle single-celled organism up to but not including multicellular life via the open source game Thrive! [Thrive]() is an evolution game akin to Spore, where the goal of the single-celled stage is to have your species survive 20 generations where you compete against different species that stem and evolve based on your original species in an ever-growing tree. The environment changes dynamically based on how life interacts with it, producing and using different resources changing which strategies work overtime. To survive a generation the population of the player species must be greater than 50, and to win at generation 20 it must be greater than 300. The single celled stage is split up between two sections: “active play” where the player plays as the micro-organism in the world, and the editor view where the player’s micro-organism can be modified between each “active play.” To skip “active play” cheat mode is activated at the beginning of the run with Infinite Compounds and Unlimit Growth Speed activated so the cell does not need to collect any compounds and can quickly re-enter the editor (simulations figure out changes in the environment and populations even if the player does nothing in “active play”.)

Figure 1: Starting point for all AI algorithms. In “active play” between editor instances.

This project will constrict the problem setting to the editor view. In the editor view, one can see the evolutionary tree that shows a tree of all the different evolution of species that evolve off of

each other (cyan nodes) starting from your species, and can die off (red nodes). It also shows the atmospheric gasses, compounds, and populations of all species as they change over time.

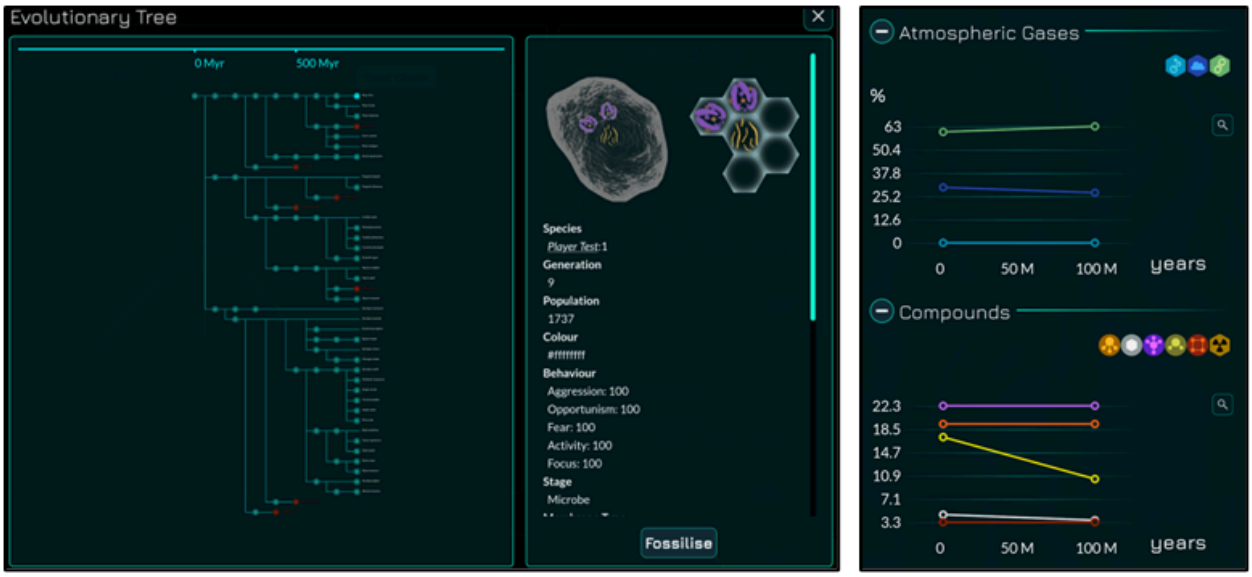

Figure 2: Evolutionary Tree and Statistics tabs in the editor view.

As this game is open source, I have modified it to output the following values to a log text file every time the editor is opened:
Ammonia, Glucose, Phosphates, Hydrogen Sulfide, Oxygen, Carbon Dioxide, Nitrogen, Sunlight, Temperature, Iron, Speed (player species/micro-organism speed), ATP Production, ATP Consumption, and Population.

These values are then converted into a unique .csv file for each of the AI algorithms to use as inputs.

To reduce complexity, the AI algorithms will be forced to stay in the starting biome (volcanic vents). There are many smaller options to tweak in the player species's Behaviour, Membrane, Tolerance, and Growth Order. But these have a small effect on the player species and are not very important if the player stays in the same biome as this problem setting will be. So, to further reduce complexity the AI algorithms will only be able to add organelles to the player species, with 12 options to pick from (excludes the nucleus and organelles that require it for now.)

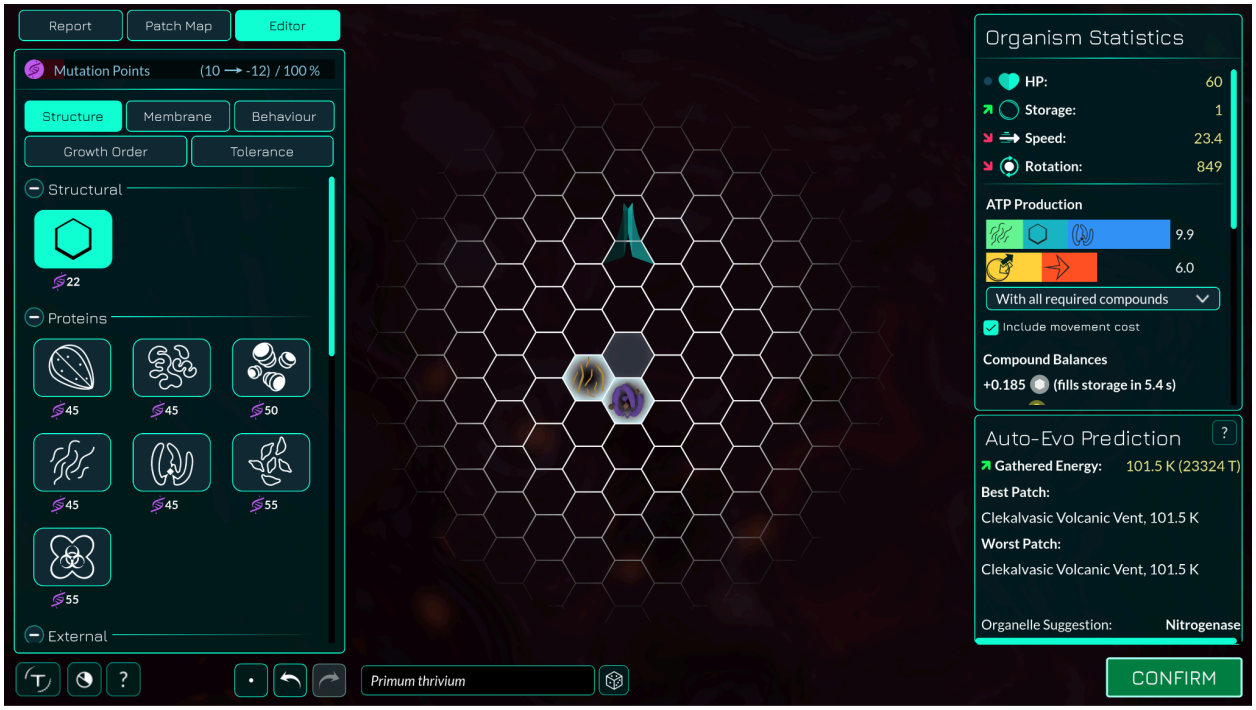

Figure 3: Main editor view where the AI algorithms will be operating.

To interact with the game, I made an interface using pyautogui to take control of the user's mouse with exact locations of each button and organelle location based on a 16:9 4K monitor. The AI algorithms can pick one organelle in each generation, placing them in a circle around the starting cytoplasm as pictured in the center of Figure 3.

The process for all three of the AI algorithms is to:

1. Enter the editor.
2. Run the AI algorithm to select what organelle to add.
3. Select said organelle and add it in a circle around the center organelle.
4. Enter "active play" (press CONFIRM), then repeat from step 1 until 20 generations have passed while the player species survived.

Numeric Planner (ENHSP)

The PDDL Planner is a Numeric Planner using ENHSP. Currently, it has a goal of producing more ATP than it consumes, which in general will allow it to survive the 20 generations. The Numeric Planner is special in that it is allowed to select multiple organelles to add as long as the total cost of the organelles added does not exceed 100. Its actions are a list of actions it will actually take in-game. Its actions consist of selecting/adding different organelles. Each action takes into account the various pros and cons of selecting each organelle, and adds the cost of the organelle to the total cost.

Bayes Net

The Bayes Net uses a Linear Gaussian Bayesian Network with dozens of edges I selected that are highly likely to affect each other. It excludes connections that while they would affect each other, the nodes had no changes in the initial dataset (would divide by zero when performing statistical calculations). The Bayes Net takes in what organelles are currently on the player species. The model is fitted and weights and biases added based on an initial dataset which consists of the initial conditions the Bayes Net will be tested on (for the first generation), and includes the results of each possible choice from the first generation. The Bayes Net makes a selection by taking in the current generation's conditions and excludes the number of organelles on the player species for each organelle, calculating what the number of organelles should be, then picking the option with the most organelles. This logic does not actually work unfortunately, as it predicts what it should be for THAT generation, not what to select for the next. I have a few ideas to try to fix this, but help here would be excellent!

Deep Learning

Trains off of a dataset of previous runs where each generation in any run is a line in deep_learning_log.csv. It uses the game's "Organelle Suggestion" as pictured in the bottom right of Figure 3 as the correct answer to refer to when training. When running it does not use the data from the run it is currently doing (though I may make it a reinforcement learning model later), and instead reads the data gathered when entering the editor to make its decision based on the model trained previously.

Deep Learning, like Bayes Net, also takes in what organelles are currently on the player species.

This Deep Learning has an option the other two algorithms do not have. It can also choose to add nothing and proceed to the next generation without performing any actions, which can be the best option at times.

Feedback Requested

1. Help with alternative options for how to get the Bayes Net to make predictions about what to select for the next generation rather than what organelles are best to be on the player species in the current generation would be amazing.
2. How can I go about importing data from the .csv file to the PDDL initial data? Do I need a separate python file to write to it? Or is there a built-in way?
3. Do I need to expand the scope of my project anymore or if I complete the Numeric Planner and fix the Bayes Net will this be acceptable? If it is acceptable I will still likely add more and improve the three methods, but I want to know where to focus my effort.